

Passive Eavesdropping Vulnerability in a batman-adv Mesh Network

SPyCy Lab - Mesh Network Vulnerability Case Study

Atena Hosseinifar | 501267191 | May 2026 | Supervisor: Dr. Navid Nasr Esfehiani, TMU

Summary

Identified and documented a passive eavesdropping vulnerability in a batman-adv wireless mesh network by capturing and analyzing live unencrypted inter-node traffic using Wireshark and a custom Scapy-based Python reporting tool, demonstrating that any node connected to the mesh could silently read all communications without detection.

Background

The SPyCy (Security and Privacy) Lab at Toronto Metropolitan University is conducting research into secure distributed systems, with a focus on threshold cryptography implemented over wireless mesh networks. The research investigates how sensitive cryptographic material, specifically shares generated by Shamir's Secret Sharing scheme, can be securely distributed and communicated across a decentralized network of nodes with no central authority.

Network Setup

- Network name (SSID): mannet-research
- Mesh protocol: batman-adv (Better Approach To Mobile Ashoc Networking)
- Mode: IBSS (ad-hoc), channel 6 (2437 MHz)
- Subnet: 10.10.0.0/16
- My node IP: 10.10.4.4/16
- Node hardware: Raspberry Pi 4 Model B
- Operating systems: Kali Linux (Node A) and Ubuntu Server 24.04.4 LTS (Node B)
- Mesh size: 5 active nodes observed during the research period
- Collaborators: Alireza Hemami (Experiment design and node management)

Methodology

The vulnerability was identified during routine mesh network monitoring as part of the warm-up research phase. While Alireza ran a scenario-driven experiment ([messenger.py](#)) that generated inter-node communication at 10-second intervals, I captured all traffic on the bat0 interface using tshark. The observation that all communications were immediately readable in plaintext was not a targeted attack, it was a routine capture that revealed the absence of any encryption layer.

Tools Used

- tshark - command-line packet capture on the bat0 mesh interface
- Wireshark - visual packet analysis and protocol dissection
- Scapy (Python) - custom pcap analysis and automated report generation
- batctl - batman-adv mesh network management and neighbor verification
- Iw / ip - wireless interface configuration and monitor mode setup

Steps

- Step 1: Joined the batman-adv mesh network by configuring wlan0 in IBSS mode and bringing up the bat0 virtual interface
- Step 2: Verified mesh connectivity using batctl n, confirmed 4 direct neighbors
- Step 3: Started tshark capture on bat0 while Alireza executed the scenario program

- Step 4: Observed that all inter-node messages appeared in plaintext in the capture output
- Step 5: Saved capture to pcap file and analyzed with custom Python/Scapy reporting tool
- Step 6: Confirmed finding across multiple experiment runs

The discovery occurred within a single lab session. The capture and analysis together took approximately 30 minutes.

The Finding

All inter-node communications on the manet-research mesh network were transmitted in plaintext with no encryption at the link layer or application layer. A passive observer connected to the mesh, or within radio range using a WiFi card in monitor mode, could capture and read all traffic without authentication, without joining the network, and without being detected.

Specifically observed:

- Application-layer messages containing cryptographic shares (the output of Shamir's Secret Sharing) transmitted as readable plaintext between nodes
- Source and destination IP addresses of all communicating nodes fully visible
- Message timing and frequency observable (one event per approximately 10 seconds, ~20 events per experiment run)
- Background mesh maintenance traffic (Gratuitous ARP, ICMPv6 Router Solicitation) also fully visible

In the context of the threshold cryptography experiment, this meant an attacker could collect enough shares from captured traffic to mathematically reconstruct the secret key, without needing to compromise any individual node, without breaking any encryption, and without generating any detectable network activity.

Evidence

The following evidence was collected:

- Live tshark output showing plaintext inter-node messages on bat0 during experiment execution
- Saved pcap capture files from multiple experiment sessions
- Output from custom Scapy-based Python reporting tool showing protocol breakdown, source IP distribution, and timing statistics

Sample reporting tool output from a representative capture:

```
=====
                        NETWORK CAPTURE REPORT
=====
Total packets captured: 100
Capture start:          2026-05-23 21:55:46
Capture end:            2026-05-23 21:57:37
Duration:               110.36 seconds

--- Protocol Breakdown ---
UDP:    46 packets
ICMP:   41 packets
Other:   13 packets

--- Top Source IPs ---
10.10.4.4:    44 packets    (my node)
10.10.1.2:    22 packets    (neighbor node)
10.10.1.3:    20 packets    (neighbor node)
=====
```

Impact

Mesh networks using batman-adv are increasingly studied for use in disaster response communications, military field networks, and decentralized infrastructure where no central router is available or trusted. These are precisely the environments where sensitive information is most likely to be transmitted and where adversaries are most likely to be physically present within radio range.

In the research context, the immediate impact is that the confidentiality goal of the threshold cryptography experiment is entirely defeated at the network layer, an attacker on the mesh can collect shares passively and reconstruct secrets without any cryptographic attack. In a real deployment scenario, this would mean:

- Any node within WiFi range, even one not connected to the mesh, could capture all traffic using a WiFi card in monitor mode
- The attacker requires no credentials, no prior knowledge of the network, and leaves no logs
- The threshold protection offered by Shamir's Secret Sharing is rendered ineffective if shares are transmitted in plaintext

Remediation

The vulnerability is architectural, it results from the absence of encryption at the link layer and application layer. Proposed remediation approaches, in order of implementation complexity:

- Application-layer encryption (most targeted): Encrypt each share before transmission using AES. The AES key itself would need to be established securely between nodes, requiring a key exchange protocol such as Diffie-Hellman or a public key infrastructure.
- Link-layer encryption: Encrypt all batman-adv traffic at the mesh layer using a shared key, protecting all communications regardless of application. This is simpler to implement but requires secure key distribution to all nodes at setup time.
- VPN overlay: Route all mesh traffic through an encrypted VPN tunnel (e.g. WireGuard), adding encryption without modifying the application or batman-adv configuration.
- Node authentication: Even with encryption, nodes should authenticate to the mesh before being allowed to participate, preventing a rogue node from joining and receiving encrypted traffic.

Connection to Cryptographic Concepts

Threshold Cryptography (Shamir's Secret Sharing)

The experiment being protected uses Shamir's Secret Sharing, a (k,n) threshold scheme where a secret is encoded as the constant term of a degree- $(k-1)$ polynomial, evaluated at n points to produce n shares. Any k shares suffice to reconstruct the polynomial and recover the secret; fewer than k shares reveal nothing mathematically.

The passive eavesdropping vulnerability directly defeats this scheme at the network level: by capturing plaintext share messages, an attacker can collect k shares and perform polynomial interpolation to recover the secret without any cryptographic weakness in the scheme itself. The math is sound; the transport is not.

AES

AES (Advanced Encryption Standard) is the likely encryption algorithm for the remediation phase. AES operates on 128-bit blocks through 10 rounds of SubBytes, ShiftRows, MixColumns, and AddRoundKey operations. Its security rests entirely on the secrecy of the key, the algorithm itself is public. Once AES encryption is applied to share transmission, captured traffic will be unreadable

without the AES key, shifting the attack surface from passive capture to key extraction or protocol weaknesses.

Lessons Learned

The most significant lesson from this work is the gap between cryptographic security and systems security. Shamir's Secret Sharing is mathematically proven secure, and yet the system as a whole was trivially vulnerable because the network transport was not considered part of the security model. Real-world security requires reasoning about every layer, not just the cryptographic primitives.